1 Lecture 18 (Oct 27) - Cycle Detection, Topological Sort, Shortest Paths, Dijkstra's Algorithm

First, finish depth-first search: detect cycles, and get a topological sort almost for free. Then start the last big problem of the algorithms part of the course: **shortest paths**.

1.1 Recap: DFS with a Clock

From Lecture 17: DFSAll unmarks every vertex, then calls DFS(v) on each vertex that is still unmarked. DFS(v) marks v, records its start time v.pre, recursively explores every unmarked out-neighbor, and records its finish time v.post. Two facts used today:

- If there is a path from v to w through vertices that are unmarked when DFS(v) starts, then DFS(v) calls DFS(w) (maybe indirectly). It may not follow that exact path.
- Sorting vertices by v.post gives **a postorder**. Different loop orders give different postorders.

An edge $u \to v$ is a **back edge** if DFS(v) (directly or indirectly) calls DFS(u). Then there is a path $v \rightsquigarrow u$, and with $u \to v$ it is a cycle.

1.2 Cycle Detection

Theorem 1. *G has a directed cycle if and only if DFSAll on G finds a back edge.*

($\Longleftarrow$): a back edge closes a cycle (Lecture 17).

($\Longrightarrow$): let C be a cycle. DFSAll calls DFS on every vertex, so let v be the **first** vertex of C that DFS is called on. Let u be the vertex just before v on C, so $u \to v$ is an edge. When DFS(v) starts, the rest of C is still unmarked, so the path around C from v to u uses only unmarked vertices. So DFS(v) calls DFS(u), and the edge $u \to v$ is a back edge.

Every cycle has **at least one** edge found as a back edge. Which one depends on the loop orders in DFSAll and DFS.

1.2.1 Spotting back edges with finish times

If $u \to v$ is a back edge, DFS(u) runs inside DFS(v):

$$v.\text{pre} < u.\text{pre} < u.\text{post} < v.\text{post}$$

You only need to compare finish times. For **any** edge $u \to v$, two other orders are impossible:

- Active intervals never partially overlap, because DFS uses a call stack.
- u.post $< v$.pre can't happen: DFS(u) sees the edge $u \to v$, so it marks v before it finishes.

So for an edge $u \to v$:

$$u \to v \text{ is a back edge} \iff u.\text{post} < v.\text{post}$$

```
IsAcyclic(G):
  DFSAll(G)                          ⟨⟨with the clock⟩⟩
  for each edge u → v
    if u.post < v.post
      return False                   ⟨⟨back edge: cycle⟩⟩
  return True
```

Running time: DFSAll marks each vertex once and looks at each edge once, and the loop is $O(1)$ per edge: $\boxed{O(|V| + |E|)}$.

Erickson's book does it another way: store a status on each vertex (new, active, finished), and report a back edge when you find an edge into an **active** vertex.

1.3 Topological Sort

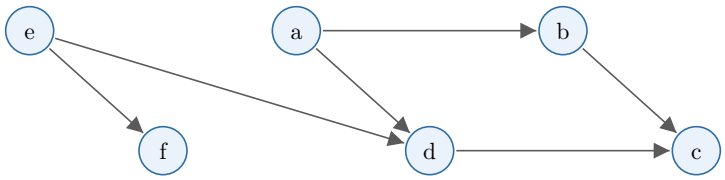
A recursive algorithm can only work if its dependency graph has no cycle. If it has none, you still need an order to solve the subproblems in: every subproblem before anything that points to it.

A **topological order** of a directed graph is an ordering of its vertices such that for every edge $u \to v$, $\boldsymbol{u}$ **comes before** $\boldsymbol{v}$. A dependency graph needs the reverse (callees first), which is a little annoying.

- **A cycle means no topological order:** Going around the cycle, some edge has to point backward.
- **No cycle: reverse a postorder:** With no cycle, DFSAll finds no back edge. So every edge $u \to v$ has v.post $< u$.post: the head finishes first. List the vertices by **decreasing** finish time, and every edge points forward.

> **Key idea** **Topological order = reversed postorder:** Run DFSAll, then list the vertices from last finished to first finished. $O(|V| + |E|)$. The same run also checks for a cycle.

Example: The Lecture 17 graph without the edge $c \to a$, so it is a DAG. DFSAll in alphabetical order gives postorder c, b, d, a, f, e. Reversed:



Topological order e, f, a, d, b, c, left to right. Every edge points right.

> **Watch out** **This is not the other topological sort algorithm** (repeatedly remove a vertex with no incoming edges). Both are correct. The DFS one goes with cycle detection and with dynamic programming on dependency graphs (Thursday).

1.4 Single-Source Shortest Paths

Input: a directed graph $G = (V, E)$, a weight function $w : E \to \mathbb{R}$, and a source vertex s. Weights may be positive, zero or even negative. Think of a road network where $w(u \to v)$ is the travel time on a stretch of road.

The **distance** $\text{dist}(s, t)$ is the minimum total weight of any path from s to t. Every known algorithm that finds one distance finds the distances from s to **every** vertex anyway: the shortest route to t needs the shortest route to the vertex before t, and so on. So the problem is **single-source shortest paths** (SSSP).

Negative weights still make sense (a scenic road you'd like to take, or the lab and homework this week). Today all weights are **positive**. Negative weights are on Thursday.

1.4.1 The shortest-path tree

A subpath of a shortest path is a shortest path: If the part from a to d weren't the shortest from a to d, replacing it with a shorter one would shorten the whole path.

So if two shortest paths split apart at a and meet again at d, both detours from a to d are shortest. Keep one of them for both paths ("uncross" them). Doing this again and again leaves a **shortest-path tree rooted at s**.

Goal: compute $\text{dist}(s, v)$ for every v, and a shortest-path tree rooted at s.

1.5 Ford's Generic Algorithm

Like whatever-first search for reachability, one general framework covers all the shortest-path algorithms. Each specific algorithm chooses an order that suits the graph's structure and weights. The framework is Lester Ford's (1956). Each vertex stores:

- v.dist: the length of **some** path from s to v, or ∞. It can be too big until the algorithm finishes.
- v.pred: the vertex before v on that path, or Null.

Following $v, v.\text{pred}, v.\text{pred.pred}, \ldots$ back to s gives a path. But it may not have length v.dist, because earlier pred values can change later. What's guaranteed: **some** path ending in $v.\text{pred} \to v$ has length v.dist.

Tense edge: $u \to v$ is **tense** if

$$u.\text{dist} + w(u \to v) < v.\text{dist}$$

That is proof you're not done: going to u and then along $u \to v$ beats the best known path to v. Fix it by **relaxing** the edge.

```
InitSSSP(s):
  s.dist ← 0
  s.pred ← Null
  for every vertex v ≠ s
```

```
    v.dist ← ∞
    v.pred ← Null

Relax(u → v):
  v.dist ← u.dist + w(u → v)
  v.pred ← u

FordSSSP(s):
  InitSSSP(s)
  while there is at least one tense edge
    Relax any tense edge
```

Claim (positive weights): FordSSSP terminates. When it does, every v.dist $= \text{dist}(s, v)$, and every v.pred is the vertex before v on a shortest path, so the pred edges form a shortest-path tree. Proof idea:

- While some v.dist is too big, some edge is tense, so it can't stop early.
- Each v.dist is always the length of a real path, and it only goes down. There are only finitely many paths, so it can't go down forever.

See Erickson's shortest-paths chapter for more detail.

> **Watch out** **Don't just say "run Ford's algorithm":** With a bad order of tense edges it can take **exponential** time: one v.dist might drop about $2^{|V|}$ times. Graders assume the worst order. Look at the graph's structure (cycles? a DAG?) and its weights (all positive? all 1? some negative?), and pick the specific algorithm that fits.

1.6 Dijkstra's Algorithm

For **positive** weights, nothing else assumed. Two observations about FordSSSP:

1. **An edge becomes tense only when its tail's dist drops:** Dist values only decrease. If $u \to v$ isn't tense, lowering v.dist can't make it tense. Only lowering u.dist can. So, like whatever-first search, revisit u's outgoing edges after u.dist drops, and not before.
2. **Relax out of the vertex with the smallest dist:** With positive weights, relaxing $u \to v$ gives v.dist $>$ u.dist. So the smallest dist value can never go down again.

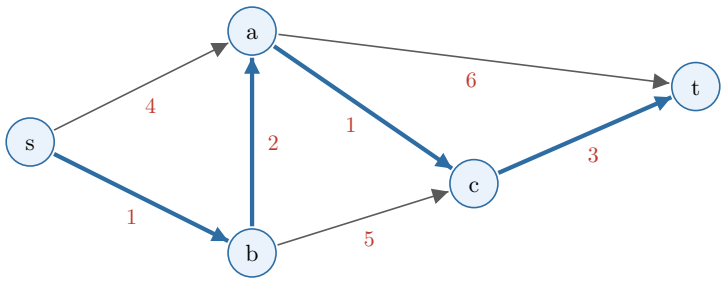
Together: keep the vertices whose dist dropped in a **priority queue**, keyed by dist.

```
Dijkstra(s):
  InitSSSP(s)
  Insert(s, 0)
  while the priority queue is not empty
    u ← ExtractMin()
    for each edge u → v
      if u → v is tense
        Relax(u → v)
```

```
if v is in the priority queue
  DecreaseKey(v, v.dist)
else
  Insert(v, v.dist)
```

The priority queue starts with only s, not every vertex. A vertex is inserted when its dist first drops. As written, a vertex could even be inserted **again** after it was extracted. With positive weights it never is (see below). Why it is written this way: Thursday.

Example: Edge weights in red. Bold edges: the final pred edges, which form the shortest-path tree:



After	s	a	b	c	t
InitSSSP	0	∞	∞	∞	∞
extract s	**0**	4	1	∞	∞
extract b		3 (via b)	**1**	6	∞
extract a		**3**		4 (via a)	9
extract c				**4**	7 (via c)
extract t					**7**

Bold: the vertex is extracted and its dist is final. The extracted dist values come out in increasing order: $0, 1, 3, 4, 7$.

1.6.1 Running time

Let d_i be the dist of the ith vertex extracted. Every new dist value created after that is d_i plus a **positive** weight, so it is bigger than d_i. Since d_i was the smallest in the queue, the extracted values **strictly increase**: $d_1 < d_2 < \ldots$

A vertex's dist never goes up. So if v were extracted twice, its second value would be no bigger than its first, contradicting the increasing order. **Each vertex is extracted at most once.** So:

- at most $|V|$ Insert and $|V|$ ExtractMin operations;
- each edge is examined at most once (when its tail is extracted), so at most $|E|$ DecreaseKey operations.

With a binary heap, each operation takes $O(\log|V|)$:

$$O((|V| + |E|)\log|V|) = \boxed{O(|E|\log|V|)}$$

The last step assumes every vertex is reachable from s, so $|E| \geq |V| - 1$. A simple graph has $|E| \leq |V|^2$, so $\log|E| \leq 2\log|V|$: $\log|E|$ and $\log|V|$ are the same in big-O.